Contents

Expressions and Assignment Statements

A Tutorial on *Concepts of Programming Languages*

This tutorial covers seven fundamental topics related to how programming languages handle expressions and assignment. Each section includes an explanation followed by code examples in common languages (C, C++, Java, Python, etc.).

---

1. Arithmetic Expressions

An **arithmetic expression** consists of *operators*, *operands*, *parentheses*, and *function calls*. The design issues for arithmetic expressions are:

- **Operator precedence rules** — which operator is evaluated first?
- **Operator associativity rules** — how are operators of equal precedence grouped?
- **Order of operand evaluation** — which operand is evaluated first?
- **Operand evaluation side effects** — does evaluating an operand change program state?
- **Operator overloading** — does an operator have multiple meanings?

Operator Precedence (typical, high to low)

Level	Operators	Description
1	`()`	Parentheses
2	** (or ^)	Exponentiation
3	`*`, `/`, `%`	Multiplicative
4	`+`, `-`	Additive

Associativity

Most binary operators are **left-associative**:

```
a - b - c   →  (a - b) - c
```

Exponentiation is usually **right-associative**:

```
a ** b ** c →  a ** (b ** c)
```

Example (C)

```c
#include <stdio.h>

int main() {
    int a = 10, b = 4, c = 2;
    int result = a + b * c;       // multiplication first  → 10 + 8 = 18
    int forced = (a + b) * c;     // parentheses override  → 14 * 2 = 28
    printf("result = %d\n", result);
    printf("forced = %d\n", forced);
    return 0;
}
```

Order of Operand Evaluation Matters

If an operand has a **side effect**, the order of evaluation can change the result.

```c
int i = 5;
int x = i + (++i);    // Undefined behavior in C!
                      // Result depends on which operand is evaluated first.
```

> **Lesson:** Avoid expressions where an operand modifies a variable used elsewhere in the same expression.

---

2. Overloaded Operators

Operator overloading lets a single operator symbol have multiple meanings depending on its operand types.

Built-in overloading

In almost every language, `+` is already overloaded: - `3 + 4` → integer addition - `3.5 + 4.2` → floating-point addition - `"Hello, " + "world"` → string concatenation (Java, Python, C++)

User-defined overloading (C++)

```cpp
#include <iostream>
using namespace std;

class Complex {
public:
    double real, imag;
    Complex(double r, double i) : real(r), imag(i) {}

    // Overload the + operator
    Complex operator+(const Complex& other) const {
        return Complex(real + other.real, imag + other.imag);
    }
};

int main() {
    Complex a(1.0, 2.0), b(3.0, 4.0);
    Complex c = a + b;        // Calls the overloaded operator+
```

```
    cout << c.real << " + " << c.imag << "i" << endl;  // 4 + 6i
    return 0;
}
```

User-defined overloading (Python)

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):           # overloads +
        return Vector(self.x + other.x, self.y + other.y)

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)        # Vector(4, 6)
```

Pros and Cons

Pros	Cons
Improves readability for ADTs	Can be abused (e.g., + doing subtraction)
Natural mathematical syntax	Hides the cost of operations
Same syntax works for built-ins and classes	Difficult to follow if heavily customized

> Java deliberately **does not allow** user-defined operator overloading (except built-in + for `String`) to keep code simple and unambiguous.

---

3. Type Conversions

A **type conversion** changes a value from one type to another. There are two kinds:

a) Widening (Implicit) Conversion — *safe*

Converts to a type that holds at least as much information.

`int → long → float → double`

b) Narrowing (Explicit) Conversion — *may lose data*

Converts to a type that may lose information.

`double → float → long → int`

Example (Java)

```java
public class TypeConversionDemo {
    public static void main(String[] args) {
        // Widening — automatic
        int i = 100;
        double d = i;             // OK: int → double
        System.out.println(d);    // 100.0

        // Narrowing — must be explicit
        double pi = 3.14159;
        int truncated = (int) pi; // cast required
        System.out.println(truncated);   // 3 (fractional part lost)
    }
}
```

Coercion vs. Casting

Term	Meaning
Coercion	Implicit conversion done automatically by the compiler
Casting	Explicit conversion requested by the programmer (`(int)x`)

Example (C)

```c
int    x = 5;
double y = 2.0;
double z = x / y;      // x is coerced to double → 2.5
double w = (double)x / y;  // explicit cast — same effect, clearer intent
```

> **Best practice:** prefer **explicit casts** to make the conversion intent obvious to readers.

---

4. Relational and Boolean Expressions

Relational Operators

These compare two operands and return a Boolean value.

Operator	C/C++/Java	Python	Pascal/Ada
Equal	`==`	`==`	`=`
Not equal	`!=`	`!=`	`<>` / `/=`
Less than	`<`	`<`	`<`
Greater than	`>`	`>`	`>`
Less or equal	`<=`	`<=`	`<=`
Greater or equal	`>=`	`>=`	`>=`

> **Note:** In C, the result of a relational expression is `1` (true) or `0` (false), not a separate Boolean type.

Boolean Operators

Operation	C/C++/Java	Python	Pascal
AND	`&&`	`and`	`and`
OR	`||`	`or`	`or`
NOT	`!`	`not`	`not`

Example (Python)

```python
age = 20
income = 35000

# Relational expressions
is_adult = age >= 18              # True
is_high_earner = income > 100000  # False

# Boolean expression combining them
eligible = is_adult and not is_high_earner
print(eligible)   # True
```

Precedence Pitfall

In most languages, **relational operators bind tighter than Boolean operators**, but Boolean operators have lower precedence than arithmetic operators:

`a + b > c && d - e < f`

is parsed as:

`((a + b) > c) && ((d - e) < f)`

> Use parentheses generously — clarity always wins over brevity.

---

5. Short-Circuit Evaluation

In a **short-circuit** Boolean evaluation, the second operand is **not evaluated** if the first operand alone determines the result.

- For `A && B`: if `A` is false → result is false, skip `B`.
- For `A || B`: if `A` is true → result is true, skip `B`.

Why It Matters

Short-circuiting is used for **guard conditions** that protect against errors:

```c
int arr[10];
int i;

// Without short-circuit: arr[i] would be accessed even if i is invalid
if (i >= 0 && i < 10 && arr[i] == 42) {
    // safe to access arr[i] here
}
```

If C did *not* short-circuit, `arr[i]` would be evaluated even when `i < 0`, causing undefined behavior.

Example (Python)

```python
def expensive_check(x):
    print("expensive_check called")
    return x > 0

# Short-circuited: expensive_check is NOT called because the first part is False
result = (False and expensive_check(5))
print(result)   # False — and "expensive_check called" is never printed

# Short-circuited: expensive_check is NOT called because the first part is True
result = (True or expensive_check(5))
print(result)   # True — and "expensive_check called" is never printed
```

Languages and Their Behavior

Language	Short-Circuit?
C, C++	&&, || short-circuit; &, | do not
Java	&&, || short-circuit; &, | do not
Python	`and`, `or` always short-circuit
Ada	`and then`, `or else` short-circuit; plain `and`, `or` do not

---

6. Assignment Statements

The **assignment statement** is the most fundamental imperative-language construct. It binds a value (right-hand side, RHS) to a variable (left-hand side, LHS).

Syntax Across Languages

Language	Syntax
C, C++, Java	`x = 10;`
Python	`x = 10`
Pascal, Ada	`x := 10;`
Fortran	`x = 10`

Pascal uses `:=` precisely so it cannot be confused with `=` (equality).

Compound Assignment Operators (C/C++/Java/Python)

```c
x += 5;     // x = x + 5
x -= 5;     // x = x - 5
x *= 2;     // x = x * 2
x /= 2;     // x = x / 2
x %= 3;     // x = x % 3
```

Multiple Assignment (Python)

```python
a, b, c = 1, 2, 3        # tuple unpacking
x = y = z = 0            # all three set to 0
a, b = b, a              # swap without a temp variable!
```

Assignment as an Expression (C / C++ / Java)

In C/C++/Java, an assignment **returns a value**, so it can appear inside another expression:

```c
int a, b, c;
a = b = c = 10;          // chained assignment — all become 10

int x;
while ((x = getchar()) != EOF) {
    // process x
}
```

In Python, assignment is a **statement**, not an expression (though `:=` — the "walrus operator" — was added in 3.8 for limited use).

List Element / Field Assignment

```python
arr = [1, 2, 3]
arr[0] = 99           # element assignment
print(arr)            # [99, 2, 3]
```

---

7. Mixed-Mode Assignment

A **mixed-mode expression** has operands of different types. A **mixed-mode assignment** is when the type of the RHS expression differs from the type of the LHS variable.

Languages Differ in How Strict They Are

Language	Mixed-mode allowed?
C / C++	Allowed; implicit conversion both ways
Java	Only **widening** allowed implicitly; narrowing needs a cast
Python	Allowed; numeric types coerced automatically
Ada	**Not** allowed — must cast explicitly

Example (C — Permissive)

```c
int    i = 10;
double d;

d = i;              // OK: int → double (widening, automatic)
i = 3.7;            // ALSO OK in C: silently truncates to 3 (warning may appear)
```

Example (Java — Stricter)

```c
int    i = 10;
double d;

d = i;              // OK: widening
// i = 3.7;         // COMPILE ERROR — narrowing not implicit
i = (int) 3.7;      // OK with explicit cast — i becomes 3
```

Example (Ada — Strictest)

```ada
I : Integer := 10;
D : Float;

-- D := I;                    -- ILLEGAL even though it's safe
D := Float(I);                -- must cast explicitly
I := Integer(3.7);            -- must cast explicitly; rounds to 4
```

Why Care?

Mixed-mode assignment is a frequent source of subtle bugs:

```c
int   total   = 7;
int   count   = 2;
double avg    = total / count;   // BUG: int/int → 3, then converts to 3.0

// Fix:
double avg2   = (double)total / count;   // 3.5 — correct
```

Rule of thumb: convert one operand *before* the operation, not the result *after* it.

Summary Table

Topic	Key Idea
Arithmetic Expressions	Precedence, associativity, and evaluation order matter
Overloaded Operators	Same symbol, different meanings — use sparingly
Type Conversions	Widening is safe and implicit; narrowing needs a cast
Relational/Boolean Exprs	Compare values; combine with &&, ||, !
Short-Circuit Evaluation	Skip the second operand when the result is already known
Assignment Statements	LHS binds to RHS value; some languages allow chaining
Mixed-Mode Assignment	RHS and LHS types differ — language rules vary widely

---

Further Reading

- Robert W. Sebesta, *Concepts of Programming Languages*, Chapter 7 — *Expressions and Assignment Statements*.
- The official language references for C, Java, Python, and Ada have detailed precedence tables and conversion rules.